

GUÍA COMPLETA DE MARKDOWN

Índice

- [GUÍA COMPLETA DE MARKDOWN](#)
- [1. TÍTULOS \(HEADINGS\)](#)
- [2. PÁRRAFOS Y SALTOS DE LÍNEA](#)
- [3. FORMATO DE TEXTO](#)
- [4. CITAS \(BLOCKQUOTES\)](#)
- [5. LISTAS](#)
 - [5.1 Listas no ordenadas](#)
 - [5.2 Listas ordenadas](#)
 - [5.3 Listas de tareas \(checkbox\)](#)
- [6. ENLACES](#)
- [7. IMÁGENES](#)
- [8. TABLAS](#)
- [9. LÍNEAS DIVISORIAS](#)
- [10. ÍNDICE AUTOMÁTICO](#)
- [11. ESCAPAR CARACTERES](#)
- [12. COMENTARIOS](#)
- [13. FOOTNOTES](#)
- [14. CÓDIGO AVANZADO](#)
- [15. BLOQUES DE ADVERTENCIA](#)
- [16. LISTAS DEFINICIONES](#)
- [17. TEXTO CENTRADO](#)
- [18. HTML DENTRO DE MARKDOWN](#)
- [19. TABLAS AVANZADAS HTML](#)
- [20. CHECKBOXES ANIDADOS](#)
- [21. REPRESENTACIÓN DE FÓRMULAS](#)
- [22. MULTIMEDIA](#)
- [23. DIAGRAMAS MERMAID](#)

Markdown es un lenguaje de marcado ligero que permite dar formato a texto de forma simple. A continuación aprenderás **toda su sintaxis**, desde lo básico hasta lo avanzado.

1. TÍTULOS (HEADINGS)

título 2

título 3

título 4

título 5

título 6

parrafo ->

Markdown soporta 6 niveles de títulos usando el símbolo #.

```
Título 1
Título 2
Título 3
Título 4
Título 5
Título 6
```

2. PÁRRAFOS Y SALTOS DE LÍNEA

Párrafo normal: solo escribe el texto.

Para un **salto de línea**, deja dos espacios al final o usa `<br>`.

Esto es un párrafo.

Esto es otra línea sin crear párrafo nuevo.

Negrita

Cursiva

Negrita y cursiva

~~function async true~~

3. FORMATO DE TEXTO

```
**Negrita**
texto en negrita

*Cursiva*
texto en cursiva

***Negrita y cursiva***
negrita y cursiva

~~Tachado~~
```

texto tachado

La función para consumir el endpoint es: `function(int x)={ cuerpo...}`

`Código en línea` : [Código en línea](#)

```
import { useRef, useState } from "react";
import { Avatar, Box, Button, Typography } from "@mui/material";
import ImageOutlinedIcon from "@mui/icons-material/ImageOutlined";

import { showToast } from "../../components/dashboard/Toast";
import { convertImageFileToWebpBase64 } from "../../utils/imageToWebp";

type Props = {
  value?: string;
  onChange: (base64Webp: string) => void;
};

/** Mismo patrón que ProductoImageField (POS Experiencias): convierte la
 * imagen a WebP en el navegador antes de guardarla. */
const MarcaImageField = ({ value, onChange }: Props) => {
  const inputRef = useRef<HTMLInputElement>(null);
  const [isConverting, setIsConverting] = useState(false);

  const handleFileChange = async (event: React.ChangeEvent<HTMLInputElement>) => {
    const file = event.target.files?.[0];
    event.target.value = "";
    if (!file) return;

    if (!file.type.startsWith("image/")) {
      showToast.error("El archivo seleccionado no es una imagen");
      return;
    }

    setIsConverting(true);
    try {
      const webpBase64 = await convertImageFileToWebpBase64(file);
      onChange(webpBase64);
    } catch (error) {
      console.error(error);
      showToast.error("No se pudo procesar la imagen");
    } finally {
      setIsConverting(false);
    }
  };

  return (
    <Box display="flex" alignItems="end" gap={2}>
      <Avatar
        src={value || undefined}
        variant="rounded"
      />
    </Box>
  );
};
```

```

        sx={{ width: 100, height: 100, bgcolor: "action.hover" }}
      >
        <ImageOutlinedIcon color="disabled" />
      </Avatar>

      <Box>
        <Button
          variant="outlined"
          size="small"
          onClick={() => inputRef.current?.click()}
          disabled={isConverting}
        >
          {isConverting ? "Procesando..." : value ? "Cambiar imagen" : "Subir
imagen"}
        </Button>
        <Typography variant="caption" color="text.secondary" display="block" mt=
{0.5}>
          Se convierte automáticamente a WebP
        </Typography>
        <input ref={inputRef} type="file" accept="image/*" hidden onChange=
{handleFileChange} />
      </Box>
    </Box>
  );
};

export default MarcaImageField;

```

texto en código:

```

Bloques de código:
```python
print("Hola mundo")

```

## 4. CITAS (BLOCKQUOTES)

Esto es una cita.

Esto es una cita con cita anidada.

Cita anidada.

## 5. LISTAS

### 5.1 Listas no ordenadas

- Elemento A
- Elemento B
  - Sub-elemento
  - Sub-sub-elemento

## 5.2 Listas ordenadas

1. Elemento 1
2. Elemento 2
3. Elemento 3
  1. Sub-elemento

## 5.3 Listas de tareas (checkbox)

- ☒ Tarea completada
- ☐ Tarea pendiente

---

# 6. ENLACES

---

Enlace simple:

```
[Texto del enlace](https://example.com)
```

Enlace con título emergente:

```
[Google](https://google.com "Ir a Google")
```

---

# 7. IMÁGENES

---

Imagen simple:

```
![Texto alternativo](ruta/imagen.png)
```

Imagen con título:

```
![Logo](logo.png "Título de imagen")
```

Imagen desde URL:

```
![Foto](https://example.com/foto.jpg)
```

## 8. TABLAS

```
| Columna A | Columna B | Columna C |
|-----|-----|-----|
| Dato 1 | Dato 2 | Dato 3 |
| Dato 4 | Dato 5 | Dato 6 |
```

## 9. LÍNEAS DIVISORIAS (HORIZONTAL RULE)

```


—
```

## 10. ÍNDICE AUTOMÁTICO (TABLE OF CONTENTS)

Los índices automáticos dependen del renderizador (GitHub, GitLab, etc.)

Ejemplo:

```
- [Título principal](#título-principal)
 - [Subtítulo](#subtítulo)
```

## 11. ESCAPAR CARACTERES

Para mostrar un símbolo sin que se interprete como formato:

```
no en cursiva
\# no es título
```

---

## 12. COMENTARIOS (NO SE VEN EN EL RENDER)

---

```
<!-- Esto es un comentario -->
```

---

## 13. FOOTNOTES (NOTAS AL PIE)

---

Compatible en muchos renderizadores:

```
Este es un texto con nota al pie[^1].

[^1]: Esta es la nota explicativa.
```

---

## 14. CÓDIGO AVANZADO

---

Activar resaltado según lenguaje:

```
```javascript  
function suma(a, b) {  
  return a + b  
}
```

15. BLOQUES DE ADVERTENCIA (GITHUB)

[!NOTE] Esto es una nota

[!WARNING] Esto es una advertencia

[!TIP] Esto es un tip

16. LISTAS DEFINICIONES

Término 1 : Definición 1

Término 2 : Definición 2

17. TEXTO CENTRADO (con HTML)

```
<p align="center" style="color:red;">Texto centrado</p>
```

Texto centrado

18. HTML DENTRO DE MARKDOWN

Markdown permite HTML:

```
<div style="color: red;">
Texto rojo
</div>
```

19. TABLAS AVANZADAS CON HTML

```
<table>
  <tr>
    <th>Col A</th>
    <th>Col B</th>
  </tr>
  <tr>
    <td>Dato 1</td>
    <td>Dato 2</td>
  </tr>
</table>
```

20. CHECKBOXES ANIDADOS

- ☐ Tarea principal
 - ☒ Subtarea 1
 - ☐ Subtarea 2

21. REPRESENTACIÓN DE FÓRMULAS (LaTeX / Math)

Inline: $a^2 + b^2 = c^2$

Bloque:

\$\$

$\int_0^1 x^2 dx$

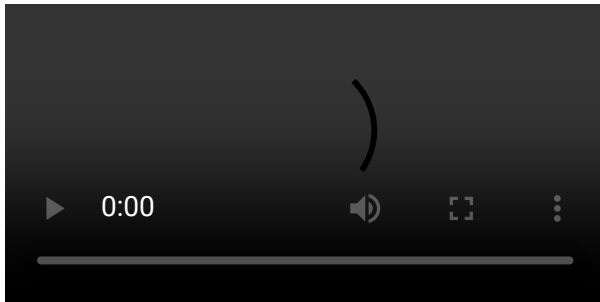
\$\$

Inline: $a^2 + b^2 = c^2$

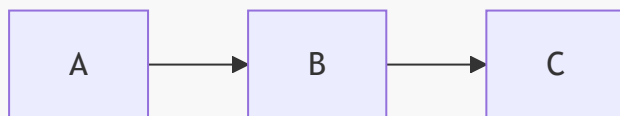
Bloque:

$$\int_0^1 x^2 dx$$

22. MULTIMEDIA (HTML)



23. DIAGRAMAS (MERMAID)



Markdown es flexible, compatible con HTML y excelente para documentación técnica, apuntes, tutoriales y README de GitHub.